



CMPT 295

Unit - Machine-Level Programming

Lecture 23 – Buffer Overflow

Last Lecture

➤ Manipulation of 1D arrays – in x86-64

- From x86-64's perspective, a 1D array is a contiguously allocated memory space of $n * L$ bytes where n is the size of the array, i.e., the number of cells where $L = \text{sizeof}(T)$ and T is the data type of the elements stored in the array
- To compute the memory address of each array element

$$\text{➤ } A[i] = \boxed{A} + \boxed{i * L}$$

Base Displacement
memory
address

where i is the index of the array element

Last Lecture

➤ Manipulation of 2D arrays – in x86-64

- From x86-64's perspective, a 2D array is a contiguously allocated memory space of $R * C * L$ bytes where R is the height (rows) and C is the width (cols) where $L = \text{sizeof}(T)$ and T is the data type of the elements stored in the array

- 2D array layout in memory: Row-Major ordering

- To compute the memory address of each row

$$A[i] : A + L(i * C)$$

- To compute the memory address of each element

$$A[i][j] : A + L(i * C + j)$$

Base memory address Displacement

Today's Menu

- Introduction
 - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
 - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – Stack
 - Overview of Function Call
 - Memory Layout and Stack - x86-64 instructions and registers
 - Passing control
 - Passing data – Calling Conventions
 - Managing local data
 - Recursion
- Array
- Buffer Overflow
- Floating-point data & operations

C and Stack ... so far

- C does not perform any bound checks on arrays
 - Local variables in C programs
 - value of Callee and caller saved registers
 - Return addresses
- } All stored on the stack
- As we saw in some of our labs and Assignment 5, this may lead to trouble

What kind of **trouble**?

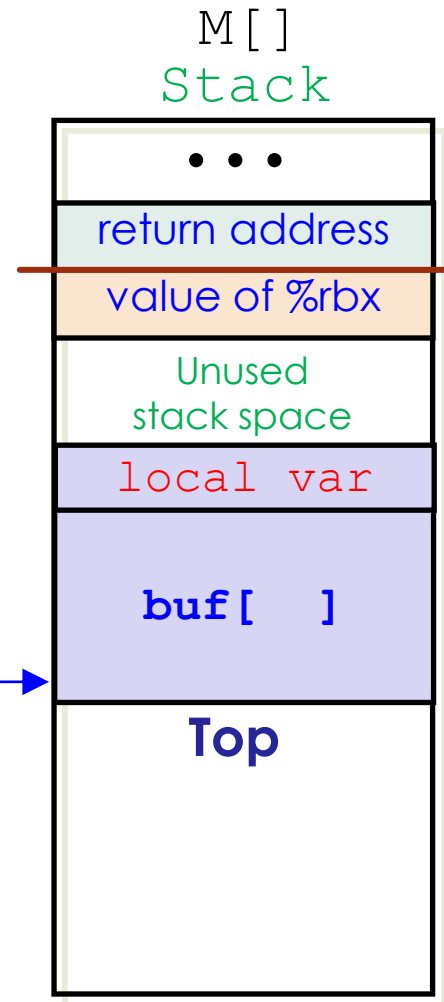
Answer: **buffer overflow** (overrun)

- When a function does not perform **bound-check** when writing to a **local array**, it could write more data into the array than the space allocated to this array can hold, hence **overflowing** the array -> **buffer overflow**

- BTW, what is a **bound-check**? Here is an example:

```
if input size <= array size
    write input into array
```

- **Effect:** the function could end up writing over, i.e., `%rsp` → **corrupting**, data kept on the **stack** such as:
 - Value of local variables and saved register values
 - Return address
- **Called:** **Stack smashing**



Demo the trouble -> *buffer overflow*

```
/*
 * Filename: password.c
 *
 * Description: Demo the effect of buffer overflow.
 *
 * Last modified on: July 2025
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
```

```
bool IsPasswordOK() {
    bool retResult = 0;
    unsigned int aSize = 9; // size of password including "\0"

    // This is the valid password.
    char goodpwd[] = "goodpass"; // goodpwd will include "\0"

    // Here is the "buf", i.e., memory storage for the password
    char password[aSize];

    // User enters password
    gets(password);

    // Echo the password
    printf("You entered the password %s\n", password);

    // strcmp(str1,str2) returns a value < 0 if str1 < str2.
    // strcmp(str1,str2) returns a value > 0 if str1 > str2.
    // strcmp(str1,str2) returns a value = 0 if str1 is equal to str2.
    //  !(0) -> !(false) = true since password equal goodpwd
    //  !(anything but 0) -> !(true) = false since Password not equal goodpwd
    return !strcmp(password, goodpwd);
}
```

```
int main() {
    int result = 0;

    puts("Enter your password:");

    // Get & check password
    if ( IsPasswordOK() ) // 1 -> true
        puts("Access granted");
    else { // 0 -> false
        puts("Access denied");
        result = 1;
    }

    return result;
}
```

Makefile

```
1  # 1) In order to (maybe) experience a "segmentation fault"
2  #    when a password exceeds the size of the buffer,
3  #    "make" the program with the flag "-fno-stack-protector".
4  #
5  # 2) In order to experience gcc's protection,
6  #    i.e., the detection of buffer overflow via
7  #    the use of canary when a password exceeds
8  #    the size of the buffer, "make clean",
9  #    comment out the SFLAGS with "-fno-stack-protector",
10 #    uncomment the SFLAGS without "-fno-stack-protector"
11 #    and "make" the program again.
12
13 SFLAGS = -S -O2 -fno-stack-protector
14 # SFLAGS = -S -O2
15 OFLAGS = -g -c
16 CFLAGS = -o pw
17
18 all: pw
19
20 pw: password.o
21     gcc $(CFLAGS) password.o
22
23 password.o: password.s
24     gcc $(OFLAGS) password.s
25
26 password.s: password.c
27     gcc $(SFLAGS) password.c
28
29 clean:
30     rm -f pw *.o password.s
```



```

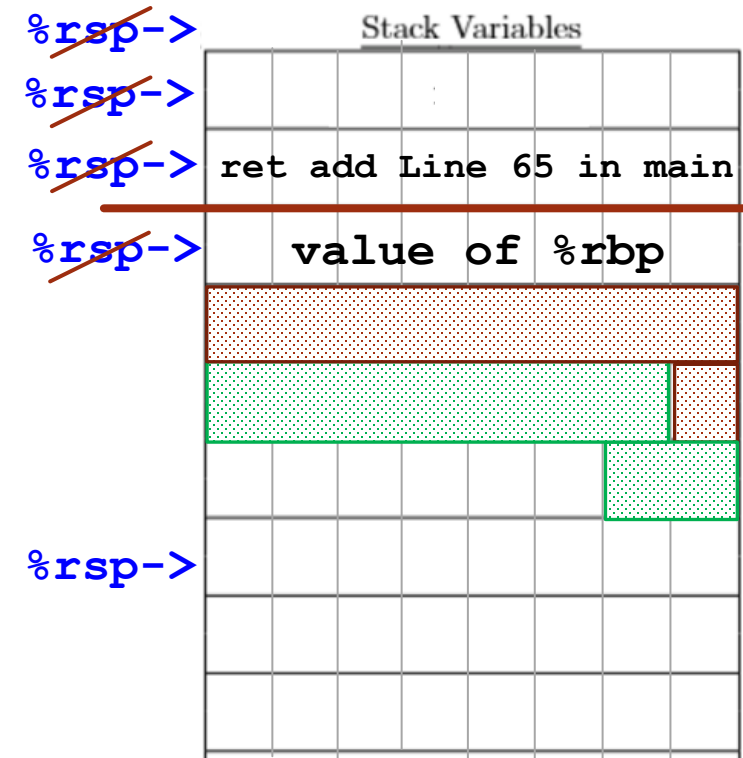
55 main:
56 .LFB52:
57     .cfi_startproc
58     endbr64
59     subq    $8, %rsp
60     .cfi_def_cfa_offset 16
61     leaq    .LC1(%rip), %rdi
62     call    puts@PLT
63     xorl    %eax, %eax
64     call    IsPasswordOK
65     testb   %al, %al
66     je      .L5
67     leaq    .LC2(%rip), %rdi
68     call    puts@PLT
69     xorl    %eax, %eax
70 .L4:
71     addq    $8, %rsp
72     .cfi_remember_state
73     .cfi_def_cfa_offset 8
74     ret
75 .L5:
76     .cfi_restore_state
77     leaq    .LC3(%rip), %rdi
78     call    puts@PLT
79     movl    $1, %eax
80     jmp     .L4

```

```

10 IsPasswordOK:
11 .LFB51:
12     .cfi_startproc
13     endbr64
14     movabsq $8319100072032694119, %rax
15     pushq   %rbp
16     .cfi_def_cfa_offset 16
17     .cfi_offset 6, -16
18     subq    $32, %rsp
19     .cfi_def_cfa_offset 48
20     leaq    23(%rsp), %rbp
21     movq    %rax, 14(%rsp)
22     xorl    %eax, %eax
23     movq    %rbp, %rdi
24     movb    $0, 22(%rsp)
25     call    gets@PLT
26     movq    %rbp, %rdx
27     movl    $1, %edi
28     xorl    %eax, %eax
29     leaq    .LC0(%rip), %rsi
30     call    __printf_chk@PLT
31     leaq    14(%rsp), %rsi
32     movq    %rbp, %rdi
33     call    strcmp@PLT
34     testl   %eax, %eax
35     sete    %al
36     addq    $32, %rsp
37     .cfi_def_cfa_offset 16
38     popq    %rbp
39     .cfi_def_cfa_offset 8
40     ret

```



```

4 .LC0:
5     .string "You entered the password %s\n"
45 .LC1:
46     .string "Enter your password:"
47 .LC2:
48     .string "Access granted"
49 .LC3:
50     .string "Access denied"

```

Why is *buffer overflow* a problem

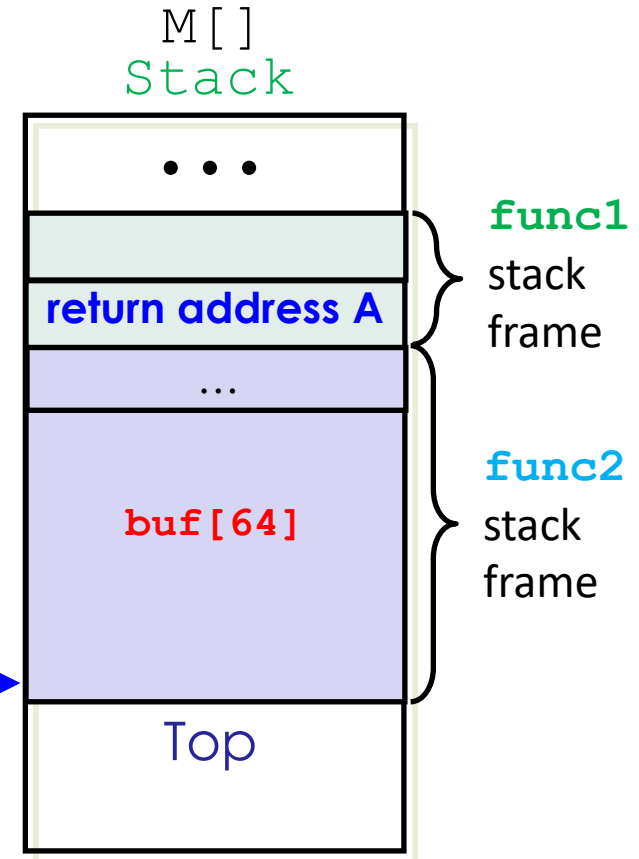
- It corrupts data
- It corrupts the return address
 - Which may lead to segmentation fault
 - Our demo desmonstrated this!
- It makes a system vulnerable to attacks
 - How?

Code injection attack

```
void func1() {  
    func2();  
    // C statement  
    at return  
    address A  
    ...  
}
```

```
int func2() {  
    char buf[64];  
    gets(buf);  
    ...  
    return ...;  
}
```

- An attacker could overflow the buffer ...
 - ... by inputting a **string** (i.e., array of chars) that contains **malicious executable code** instead of legitimate characters
 - The **string** is written to array **buf** on **stack** and overwrites **return address A** with a **return address** that points to **malicious executable code** somewhere in **buf**
 - When **func2** executes **ret** instruction, **%rsp** → it pops this erroneous **return address** onto PC (**%rip**) and the microprocessor fetches the **malicious instruction** at PC (**%rip**), decodes it and starts executing this **malicious instruction** along with the rest of the **malicious executable code** in **buf**



How to protection against such *attack*

1. Avoid creating **overflow vulnerabilities** in the code that we write by always
 - Doing **bound-checks** on arrays
 - Calling C library functions that limit string lengths
 - **Unsafe:** `gets()`, `strcpy()`, `strcat()`, `sprintf()`, ...
 - These functions can generate a byte sequence without being given any indication of the size of the destination buffer
 - **Safe:** `fgets()`

Suggestion from **developer.apple.com**

Copies up to **sizeof(destination)** -> **n** characters from the string pointed to by **source** to **destination**. In the case where the length of **source** is less than **n**, the remainder of **destination** will be padded with null bytes. In the case where the length of **source** is greater than **n**, the **destination** will contain a truncated version of **source**.

```
char destination[5];  
char * source = "LARGER";
```

```
strcpy(destination, source);
```



Copies the string pointed to by **source** (including the null character) to the **destination** and returns it.
=> Buffer overflow

```
strncpy(destination, source, sizeof(destination));
```



```
strncpy(destination, source, sizeof(destination));
```



Copies up to **sizeof(destination) - 1** -> **n - 1** characters from null-terminated **source** to **destination**, it then "null" terminates **destination** and returns the length of **source**.

<https://linux.die.net/man/3/strncpy>

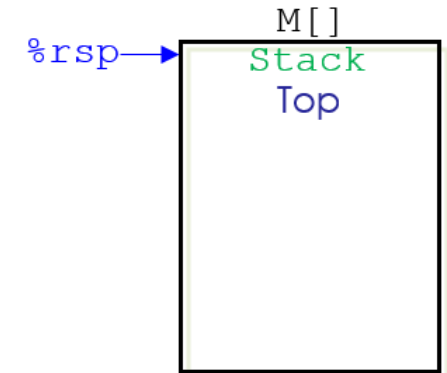
How to protection against such *attack*

2. Employ system-level protections

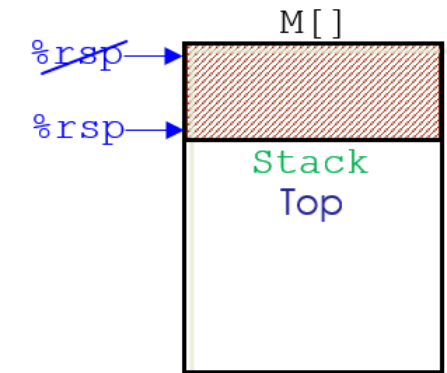
A. Randomized *stack* offsets

- When a program starts executing, the system first allocates a random amount of space on *stack* then it allocates a stack frame to the program's **main()** function
- **Effect:** Shifts *stack* addresses (`%rsp`) for entire program
 - Shifts the memory address of all the stack frames allocated to program's functions when they are called
- Hence, makes it difficult for hackers to predict the start memory address of each stack frame where **malicious executable code** may have been inserted and therefore to predict the return address to the **malicious executable code** since stack is *repositioned* each time program executes

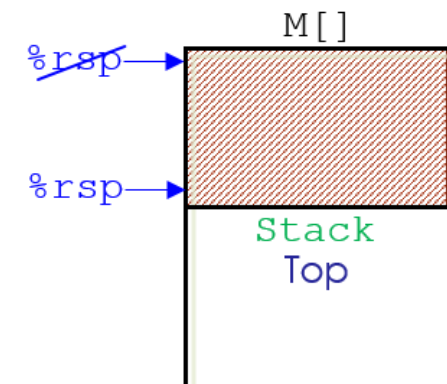
So, instead of this ...



... the stack may be like this ...



or that ...

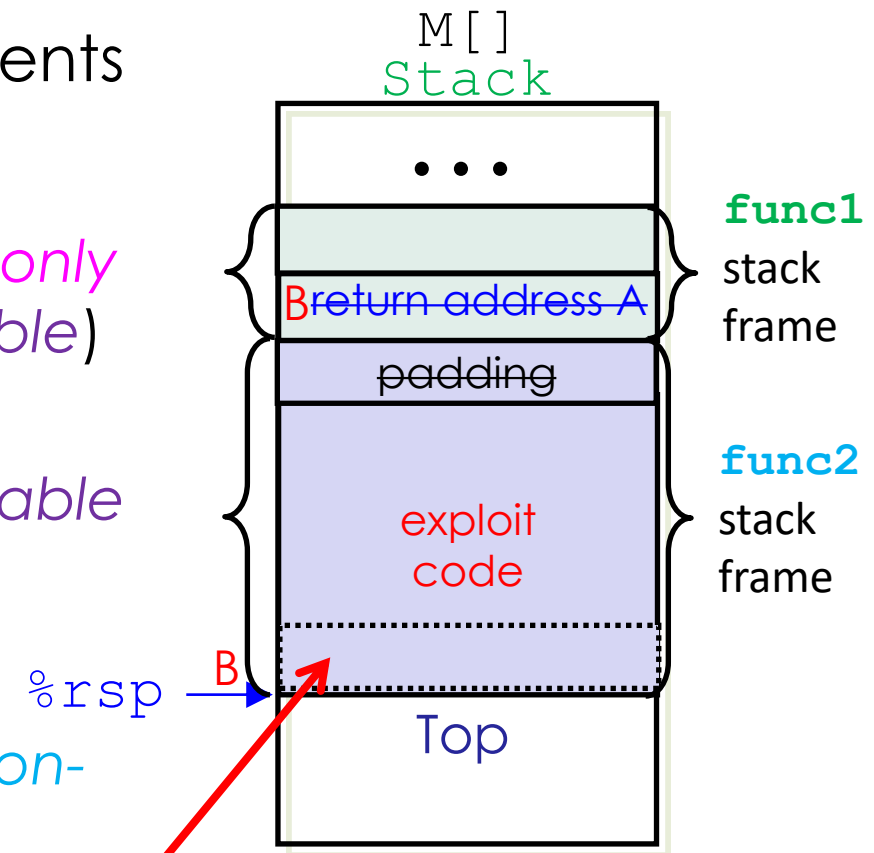


How to protection against such *attack*

2. Employ system-level protections

B. Non-executable code segments

- In the old days of **x86**, memory segments marked as either *read-only* or *writable* (both implied *readable*)
=> 2 types of permissions
 - Could execute anything *readable*
- **x86-64** has added an explicit *executable* permission
- **Stack** segment now marked as *non-executable*



Any attempt to execute this code will fail

How to protection against such *attack*

3. Compiler (like gcc) uses a *stack canary* value

- History: Starting early 1900's, canaries were used in coal mines to detect gas leaks
- Push a randomized *canary* value between an array and return address on stack (remember our Lab 8 and Assignment 5)
- Before executing a *ret* instruction, *canary* value is checked to see if it has been corrupted
 - If so, failure is reported and program aborts

```
main:  # main.c from our Lab 4
endbr64
pushq %rbp
...
subq  $64, %rsp
movq  %fs:40, %rax
movq  %rax, 56(%rsp)
...
leaq  16(%rsp), %rbp
...
movq  56(%rsp), %rax
xorq  %fs:40, %rax
jne   .L5
addq  $64, %rsp
popq  %rbp
ret
.L5:
call  __stack_chk_fail@PLT
```


Makefile

```
1 # 1) In order to (maybe) experience a "segmentation fault"
2 #   when a password exceeds the size of the buffer,
3 #   "make" the program with the flag "-fno-stack-protector".
4 #
5 # 2) In order to experience gcc's protection,
6 #   i.e., the detection of buffer overflow via
7 #   the use of canary when a password exceeds
8 #   the size of the buffer, "make clean",
9 #   comment out the SFLAGS with "-fno-stack-protector",
10 #   uncomment the SFLAGS without "-fno-stack-protector"
11 #   and "make" the program again.
12
13 # SFLAGS = -S -O2 -fno-stack-protector
14 SFLAGS = -S -O2
15 OFLAGS = -g -c
16 CFLAGS = -o pw
17
18 all: pw
19
20 pw: password.o
21     gcc $(CFLAGS) password.o
22
23 password.o: password.s
24     gcc $(OFLAGS) password.s
25
26 password.s: password.c
27     gcc $(SFLAGS) password.c
28
29 clean:
30     rm -f pw *.o password.s
```

Demo – Canary

```
64 main:
65 .LFB52:
66     .cfi_startproc
67     endbr64
68     subq    $8, %rsp
69     .cfi_def_cfa_offset 16
70     leaq    .LC1(%rip), %rdi
71     call    puts@PLT
72     xorl    %eax, %eax
73     call    IsPasswordOK
74     testb   %al, %al
75     je      .L7
76     leaq    .LC2(%rip), %rdi
77     call    puts@PLT
78     xorl    %eax, %eax
79 .L6:
80     addq    $8, %rsp
81     .cfi_remember_state
82     .cfi_def_cfa_offset 8
83     ret
84 .L7:
85     .cfi_restore_state
86     leaq    .LC3(%rip), %rdi
87     call    puts@PLT
88     movl    $1, %eax
89     jmp     .L6
```

```
10 IsPasswordOK:
11     .LFB51:
12     .cfi_startproc
13     endbr64
14     pushq   %rbp
15     .cfi_def_cfa_offset 16
16     .cfi_offset 6, -16
17     subq    $32, %rsp
18     .cfi_def_cfa_offset 48
19     movq    %fs:40, %rax
20     movq    %rax, 24(%rsp)
21     movabsq $8319100072032694119, %rax
22     leaq    6(%rsp), %rbp
23     movb    $0, 23(%rsp)
24     movq    %rbp, %rdi
25     movq    %rax, 15(%rsp)
26     xorl    %eax, %eax
27     call    gets@PLT
28     movq    %rbp, %rdx
29     movl    $1, %edi
30     xorl    %eax, %eax
31     leaq    .LC0(%rip), %rsi
32     call    __printf_chk@PLT
33     leaq    15(%rsp), %rsi
34     movq    %rbp, %rdi
35     call    strcmp@PLT
36     testl   %eax, %eax
37     sete    %al
38     movq    24(%rsp), %rdx
39     subq    %fs:40, %rdx
40     jne     .L5
41     addq    $32, %rsp
42     .cfi_remember_state
43     .cfi_def_cfa_offset 16
44     popq    %rbp
45     .cfi_def_cfa_offset 8
46     ret
```

~~%rsp->~~

~~%rsp->~~

~~%rsp->~~

~~%rsp->~~

%rsp->

Stack Variables

ret add Line 74 in main							
value of %rbp							
canary value							
o o d p a s s 0							
							g

4

.LC0:

5

.string "You entered the password %s\n"

54

.LC1:

55

.string "Enter your password:"

56

.LC2:

57

.string "Access granted"

58

.LC3:

59

.string "Access denied"

How to protection against such *attack*

3. Newest version of our gcc compiler (version 8 and up) uses **Control-Flow Enforcement Technology** (CET)

- Instruction **endbr64** (End Branch 64 bit)
-> Terminate Indirect Branch in 64 bit
- Compiler inserts **endbr64** at all valid destinations of control transfers
- Microprocessor tracks indirect branching and ensures that all indirect branching instructions lead to valid functions starting with **endbr64**
 - After an indirect **jmp** or **call**, the next instruction is expected to be **endbr64**
 - If so -> microprocessor infers that function is **safe** to execute
 - If not -> microprocessor infers that control flow may have been manipulated by some **malicious code**, i.e., function is **unsafe** to execute and **aborts!**

```
main:
.LFB23:
    .cfi_startproc
    endbr64
    pushq    %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    movl     $9, %ecx
    movl     $6, %edx
    ...
```

Examples of indirect branching instruction in x86-64:

```
call    *%rbx
jmp     *0x4bca2(%rip)
```

Source: <https://stackoverflow.com/questions/56905811/what-does-the-endbr64-instruction-actually-do>

From
stack
overflow



Storing Data in Various Segments of Memory - Optional

Where can we store our data

The material on the next 3 slides is optional
-> It is for your learning pleasure!

We have already covered the idea of storing data in registers, on the **stack** and on the **heap**.

- Data in registers
- Data on the **stack** memory (on **stack frame** of a function)
 - This memory is used temporarily while a function executes
 - And then, once the function terminates, it is recycled
- Data on the **heap** memory
 - This memory is allocated (temporarily) when a function calls **malloc(...)** (and other similar functions)
 - And then, once free'd, this memory is also recycled
- Data in **fixed** memory, i.e., **Data segment**
 - Statically allocated data
 - e.g., global variables, static variables, string constants
 - Lasts while the whole program executes

What does this type of data look like?

Data stored in Data Segment of memory

➤ Declared using:

1. label which is a **memory address**
2. size directive: ¹.byte, ².word, ⁴.long, ⁸.quad
3. initial value

➤ **Example 1:**

C:

```
long x = 6;  
long y = 9;  
void main {  
    . . .  
}
```

x86-64:

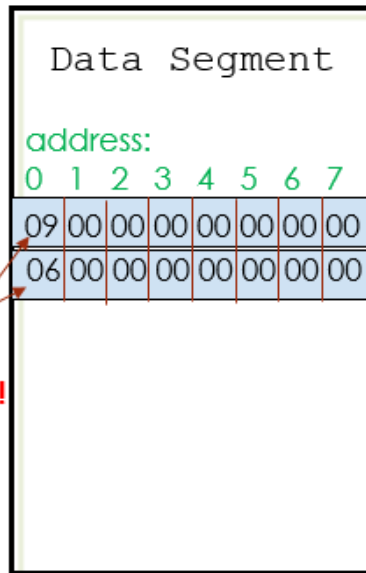
```
x: .quad 6 # 0x0000000000000006  
y: .quad 9 # 0x0000000000000009
```

label

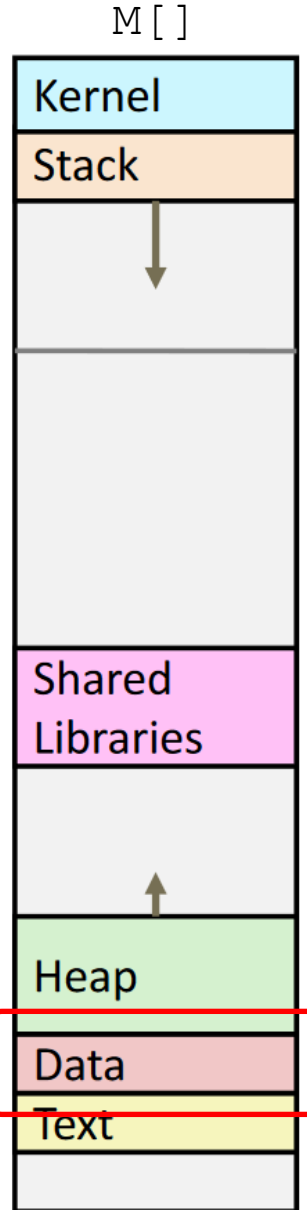
size directive

initial value

M[]



LSB
Remember:
little endian!



Data stored in Data Segment – Example 2

```
.LC0:
    .string "The total is %d.\n"
main:
.LFB38:
    .cfi_startproc
    subq    $8, %rsp
    .cfi_def_cfa_offset 16
    movl    $6, %esi
    movl    $A, %edi
    call    sum_array
    movl    $.LC0, %esi
    movl    %eax, %edx
    movl    $1, %edi
    xorl    %eax, %eax
    addq    $8, %rsp
    .cfi_def_cfa_offset 8
    jmp     __printf_chk
```

```
#define N 6

int A[N] = {12, 34, 56, 78, -90, 1};

void main () {

    printf("The total is %d.\n", sum_array(A,N));

    return;
}
```

```
A:    . . .    #
      .long    12        // or .long 12, 34, 56, 78, -90, 1
      .long    34
      .long    56
      .long    78
      .long    -90
      .long    1
```


Data stored on Stack

- **Example 1**

```
void main( int argc, char * argv ) {  
  
    int A[ ] = {12, 34, 56, 78, -90, 1};  
    printf("The total is %d.\n", sum_array(A,N));  
    return;  
}
```

```
main:
.LFB38:
.cfi_startproc
subq    $40, %rsp
.cfi_def_cfa_offset 48
movl    $6, %esi
movq    %fs:40, %rax
movq    %rax, 24(%rsp)
xorl    %eax, %eax
movabsq $146028888076, %rax
movq    %rsp, %rdi
movq    %rax, (%rsp)
movabsq $335007449144, %rax
movq    %rax, 8(%rsp)
movabsq $8589934502, %rax
movq    %rax, 16(%rsp)
call    sum_array
movl    $.LC0, %esi
movl    %eax, %edx
movl    $1, %edi
xorl    %eax, %eax
call    __printf_chk
movq    24(%rsp), %rax
xorq    %fs:40, %rax
jne     .L5
addq    $40, %rsp
.cfi_remember_state
.cfi_def_cfa_offset 8
ret
```

This is how this large number ends up representing 12 and 34:

- [illegible]

Summary

- What is a **buffer overflow**
 - **Defn**: When a function writes more data in array on the **stack** than the array can hold
 - **Effect**: Data kept on the **stack** (value of other local variables, saved register values and return address) may be **corrupted**
- > **Stack smashing**
- Why does buffer overflow spell **trouble**?
Because it creates vulnerability by allowing **hacker attacks**
- How to protect system against such attacks
 1. Avoid creating **overflow vulnerabilities** in the code that we write
 - By always checking array bounds and calling “safe” library functions that consider size of array
 2. Employ system-level protections
 - Randomized initial **stack** pointer and non-executable code segments
 3. Use compiler (like `gcc`) security features:
 - **Stack canary** value and **endbr64** instruction

s/w developer

system

compiler

Next Lecture

- Brief look at floating point data and operations
- Start a new unit ...
 - Instruction Set Architecture (ISA)